

AI-Assisted Coding Lab Assignment=12.1

Name: Pothunuri Akshitha

Roll no: 2403A510C2

Batch no: 05

CSE 2nd year

Task Description #1 (Sorting – Merge Sort Implementation)

- Task: Use AI to generate a Python program that implements the Merge Sort algorithm.

Prompt:

create a Python function called `merge_sort(arr)` that takes a list `arr` of numbers and returns a new list sorted in ascending order using the Merge Sort algorithm.

- Include a detailed docstring for the function that explains:
 - What the function does.
 - The time complexity of Merge Sort.
 - The space complexity of Merge Sort.
- Make sure the code is clean, well-commented, and easy to understand.
- After the function definition, provide some example test cases to verify that the implementation works correctly.

Code Generated:

12.1.task1.py > ...

```
1 def merge_sort(arr):
2     """
3     Sorts a list of numbers in ascending order using the Merge Sort algorithm.
4
5     Merge Sort is a divide-and-conquer algorithm that:
6     - Divides the list into two halves,
7     - Recursively sorts each half,
8     - Merges the two sorted halves into a single sorted list.
9
10    Time Complexity:
11    | - Best, Average, and Worst case:  $O(n \log n)$ , where  $n$  is the number of elements in the list.
12
13    Space Complexity:
14    | -  $O(n)$ , due to the temporary arrays created during the merge process.
15
16    Parameters:
17    arr (list): A list of numbers to be sorted.
18
19    Returns:
20    list: A new list containing all elements from `arr` in ascending order.
21    """
22    if len(arr) <= 1:
23        return arr
24
25    # Divide the list into two halves
26    mid = len(arr) // 2
27    left_half = merge_sort(arr[:mid])
28    right_half = merge_sort(arr[mid:])
29
30    # Merge the sorted halves
31    return merge(left_half, right_half)
32
33 def merge(left, right):
34     """
35     Merges two sorted lists into one sorted list.
36
37     Parameters:
```

```

36
37     Parameters:
38     left (list): Sorted list.
39     right (list): Sorted list.
40
41     Returns:
42     list: Merged and sorted list.
43     """
44     merged = []
45     i = j = 0
46
47     # Compare elements from left and right lists and append the smaller one
48     while i < len(left) and j < len(right):
49         if left[i] <= right[j]:
50             merged.append(left[i])
51             i += 1
52         else:
53             merged.append(right[j])
54             j += 1
55
56     # Append any remaining elements from left or right
57     merged.extend(left[i:])
58     merged.extend(right[j:])
59
60     return merged
61
62
63     # Test cases to verify the implementation
64     if __name__ == "__main__":
65         test_cases = [
66             [],
67             [1],
68             [5, 3, 8, 4, 2],
69             [10, 9, 8, 7, 6, 5],
70             [3, 3, 3, 3],
71             [100, -1, 50, 23, 0, -99]
72         ]
73
74         for i, test in enumerate(test_cases, 1):
75             sorted_list = merge_sort(test)
76             print(f"Test case {i}: Original: {test} -> Sorted: {sorted_list}")
77

```

Output:

```

[Running] python -u "c:\Users\akshi\OneDrive\Desktop\AI_Assisted_Coding\12.1.task1.py"
Test case 1: Original: [] -> Sorted: []
Test case 2: Original: [1] -> Sorted: [1]
Test case 3: Original: [5, 3, 8, 4, 2] -> Sorted: [2, 3, 4, 5, 8]
Test case 4: Original: [10, 9, 8, 7, 6, 5] -> Sorted: [5, 6, 7, 8, 9, 10]
Test case 5: Original: [3, 3, 3, 3] -> Sorted: [3, 3, 3, 3]
Test case 6: Original: [100, -1, 50, 23, 0, -99] -> Sorted: [-99, -1, 0, 23, 50, 100]

[Done] exited with code=0 in 0.181 seconds

```

Observation:

The Merge Sort algorithm efficiently sorts a list by recursively dividing it into smaller sublists until single-element lists remain. It then merges these sorted sublists to produce a fully sorted list. This divide-and-conquer approach ensures a consistent time complexity of $O(n \log n)$.

$\log n$) across best, average, and worst cases, making it reliable for large datasets. However, it requires additional space proportional to the input size, leading to a space complexity of $O(n)$. The implementation handles edge cases like empty or single-element lists gracefully. Merge Sort is stable, meaning it preserves the relative order of equal elements. Its recursive nature may lead to higher memory usage compared to in-place algorithms. Overall, it balances time efficiency with manageable space overhead, suitable for a wide range of sorting tasks.

Task Description #2 (Searching – Binary Search with AI Optimization)

- Task: Use AI to create a binary search function that finds a target element in a sorted list

Prompt:

Please create a Python function called `binary_search(arr, target)` that searches for a target element in a sorted list `arr`. The function should return the index of the target if found, or `-1` if the target is not in the list.

- Include a detailed docstring that explains:
 - What the function does.
 - The best-case, average-case, and worst-case time complexities of binary search.
- Add clear, AI-generated comments in the code to explain each step.
- Provide example test cases after the function to demonstrate and verify the implementation with different inputs.

Code Generated:

```

12.2.task2.py > ...
1 def binary_search(arr, target):
2     """
3     Performs binary search to find the index of `target` in a sorted list `arr`.
4
5     Binary Search works by repeatedly dividing the search interval in half:
6     - Compare the target with the middle element.
7     - If they are equal, return the index.
8     - If the target is smaller, continue searching in the left half.
9     - If the target is larger, continue searching in the right half.
10    - Repeat until the target is found or the interval is empty.
11
12    Time Complexity:
13    - Best case: O(1) when the target is at the middle element.
14    - Average case: O(log n), where n is the number of elements in the list.
15    - Worst case: O(log n), when the target is not found or at the edges.
16
17    Space Complexity:
18    - O(1), as it uses constant extra space.
19
20    Parameters:
21    arr (list): A sorted list of elements to search.
22    target (any): The element to search for.
23
24    Returns:
25    int: The index of the target element if found; otherwise, -1.
26    """
27    left, right = 0, len(arr) - 1 # Initialize pointers to the start and end of the list
28
29    while left <= right:
30        mid = (left + right) // 2 # Find the middle index
31
32        # Check if the target is at the middle
33        if arr[mid] == target:
34            return mid
35
36        # If target is smaller, discard the right half
37        elif target < arr[mid]:
38            right = mid - 1
39
40        # If target is larger, discard the left half
41        else:
42            left = mid + 1
43
44    return -1 # Target was not found in the list
45
46
47 # Example test cases to verify the implementation
48 if __name__ == "__main__":
49     test_cases = [
50         ([], 5),
51         ([1], 1),
52         ([1, 2, 3, 4, 5], 3),
53         ([10, 20, 30, 40, 50], 10),
54         ([10, 20, 30, 40, 50], 60),
55         ([5, 10, 15, 20, 25, 30], 25)
56     ]
57
58     for i, (arr, target) in enumerate(test_cases, 1):
59         result = binary_search(arr, target)
60         print(f"Test case {i}: Searching {target} in {arr} -> Index found: {result}")
61

```

Output:

```
[Running] python -u "c:\Users\akshi\OneDrive\Desktop\AI_Assisted_Coding\12.2.task2.py"
Test case 1: Searching 5 in [] -> Index found: -1
Test case 2: Searching 1 in [1] -> Index found: 0
Test case 3: Searching 3 in [1, 2, 3, 4, 5] -> Index found: 2
Test case 4: Searching 10 in [10, 20, 30, 40, 50] -> Index found: 0
Test case 5: Searching 60 in [10, 20, 30, 40, 50] -> Index found: -1
Test case 6: Searching 25 in [5, 10, 15, 20, 25, 30] -> Index found: 4

[Done] exited with code=0 in 0.138 seconds
```

Observation:

Binary search efficiently locates a target element in a sorted list by repeatedly halving the search range, leading to a logarithmic time complexity of $O(\log n)$. The best case occurs when the target is at the middle element, resulting in $O(1)$ time. This method uses constant extra space, making it space-efficient with $O(1)$ complexity. Binary search requires the input list to be sorted beforehand for correct operation. If the target is not found, the function returns -1, indicating absence. The iterative approach avoids the overhead of recursive calls. It performs well on large datasets due to its fast search capability. Overall, binary search is a fundamental and highly efficient algorithm for search tasks in sorted collections.

Task Description #3 (Real-Time Application – Inventory Management System)

- Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:
 1. Quickly search for a product by ID or name.
 2. Sort products by price or quantity for stock analysis.
- Task:
 - o Use AI to suggest the most efficient search and sort algorithms for this use case.
 - o Implement the recommended algorithms in Python.
 - o Justify the choice based on dataset size, update frequency, and performance requirements.

Prompt:

Design and implement an inventory management system for a retail store with thousands of products. Each product has attributes like product ID, name, price, and stock quantity.

- 1. Suggest the most efficient search algorithms for quickly finding products by ID or name.**
- 2. Suggest the most efficient sorting algorithms for sorting products by price or quantity.**
- 3. Provide a clear justification for each recommended algorithm, considering factors like dataset size, update frequency, and performance requirements.**

4. Implement the recommended search and sort algorithms in Python as functions that operate on a list of product dictionaries or objects.
5. Include at least 3 assert test cases for each function to verify correctness.
6. Present a summary table mapping each operation (search by ID, search by name, sort by price, sort by quantity) to the recommended algorithm and the justification for choosing it.

Please make sure the code is clean, well-commented, and easy to understand.

Code Generated:

```

12.1.task3.py > ...
1  # Inventory Management System: Search and Sort
2
3  # Sample product data structure:
4  # Each product is a dict with keys: 'id', 'name', 'price', 'quantity'
5
6  # Recommended Algorithms:
7  # | Operation      | Algorithm      | Justification
8  # |-----|-----|-----|
9  # | Search by ID    | Hash Table (dict) | O(1) average lookup; fast direct access by unique key (ID).
10 # | Search by Name   | Binary Search    | Requires sorting by name; O(log n) search, efficient for many queries on static or less-frequent updates.
11 # | Sort by Price    | TimSort (Python's built-in sorted()) | Stable, O(n log n) average & worst-case, efficient for real-world data.
12 # | Sort by Quantity | TimSort          | Same reasoning as price sorting; built-in sorting is highly optimized.
13
14 # Implementation:
15
16 def build_id_index(inventory):
17     """
18     Build a dictionary mapping product IDs to product info for O(1) lookup.
19     """
20     return {product['id']: product for product in inventory}
21
22 def binary_search_by_name(inventory, target_name):
23     """
24     Search for a product by name using binary search.
25     Assumes inventory is sorted by 'name' attribute.
26     Returns the index of the product or -1 if not found.
27     """
28     left, right = 0, len(inventory) - 1
29     while left <= right:
30         mid = (left + right) // 2
31         mid_name = inventory[mid]['name']
32         if mid_name == target_name:
33             return mid
34         elif target_name < mid_name:
35             right = mid - 1
36         else:
37             left = mid + 1
38     return -1
39
40 def sort_inventory_by_price(inventory):
41     """
42     Sorts the inventory by product price in ascending order using Python's built-in sorted (TimSort).
43     """
44     return sorted(inventory, key=lambda x: x['price'])
45
46 def sort_inventory_by_quantity(inventory):
47     """
48     Sorts the inventory by stock quantity in ascending order using Python's built-in sorted (TimSort).
49     """
50     return sorted(inventory, key=lambda x: x['quantity'])

```

```

51
52
53 # Sample inventory for testing
54 inventory = [
55     {'id': 'p1003', 'name': 'Apple', 'price': 0.5, 'quantity': 120},
56     {'id': 'p1001', 'name': 'Banana', 'price': 0.3, 'quantity': 200},
57     {'id': 'p1002', 'name': 'Carrot', 'price': 0.4, 'quantity': 150},
58     {'id': 'p1004', 'name': 'Date', 'price': 1.5, 'quantity': 50},
59 ]
60
61 # Build ID index for fast lookup by ID
62 id_index = build_id_index(inventory)
63
64
65 # ----- Test cases -----
66
67 def test_search_by_id():
68     assert id_index['p1001']['name'] == 'Banana'
69     assert id_index.get('p9999') is None # Non-existent ID
70
71 def test_search_by_name():
72     sorted_by_name = sorted(inventory, key=lambda x: x['name'])
73     idx = binary_search_by_name(sorted_by_name, 'Carrot')
74     assert idx != -1 and sorted_by_name[idx]['id'] == 'p1002'
75
76     idx = binary_search_by_name(sorted_by_name, 'Eggplant')
77     assert idx == -1
78
79 def test_sort_by_price():
80     sorted_price = sort_inventory_by_price(inventory)
81     prices = [p['price'] for p in sorted_price]
82     assert prices == sorted(prices)
83
84 def test_sort_by_quantity():
85     sorted_quantity = sort_inventory_by_quantity(inventory)
86     quantities = [p['quantity'] for p in sorted_quantity]
87     assert quantities == sorted(quantities)
88
89 if __name__ == "__main__":
90     # Run test cases
91     test_search_by_id()
92     test_search_by_name()
93     test_sort_by_price()
94     test_sort_by_quantity()
95
96     print("All test cases passed!")
97

```

Output:

```

[Running] python -u "c:\Users\akshi\OneDrive\Desktop\AI_Assisted_Coding\12.2.task2.py"
Test case 1: Searching 5 in [] -> Index found: -1
Test case 2: Searching 1 in [1] -> Index found: 0
Test case 3: Searching 3 in [1, 2, 3, 4, 5] -> Index found: 2
Test case 4: Searching 10 in [10, 20, 30, 40, 50] -> Index found: 0
Test case 5: Searching 60 in [10, 20, 30, 40, 50] -> Index found: -1
Test case 6: Searching 25 in [5, 10, 15, 20, 25, 30] -> Index found: 4

[Done] exited with code=0 in 0.138 seconds

[Running] python -u "c:\Users\akshi\OneDrive\Desktop\AI_Assisted_Coding\12.1.task3.py"
All test cases passed!

[Done] exited with code=0 in 0.135 seconds

```

Observation:

Using a hash table (Python dict) for searching by product ID provides constant-time $O(1)$ lookup, ideal for unique keys and frequent queries. Searching by name uses binary search on a sorted list, offering $O(\log n)$ efficiency but requiring data to be pre-sorted. Sorting by price or quantity leverages Python's built-in TimSort, which is stable and performs well on real-world datasets with $O(n \log n)$ time complexity. This combination balances fast searches and efficient sorting for large inventories. The choice suits thousands of products with moderate update frequency. Hash tables handle dynamic updates well, while binary search is best for less frequent name searches. Overall, the system ensures quick access and flexible sorting needed for inventory management.